

Playwright Automation — Chapitre 2

Locators • Actions • Assertions • Collections • Attente dynamique

Documentation suivant la progression et les exemples du cours, avec des explications pédagogiques supplémentaires clairement signalées lorsqu'elles vont au-delà des notes.

1. Objectifs du chapitre

- Identifier des éléments avec les locators Playwright.
- Utiliser des sélecteurs CSS et XPath.
- Remplir des champs et cliquer sur des éléments.
- Extraire du texte depuis une page.
- Écrire des assertions avec expect().
- Utiliser test.only() pour isoler un test.
- Travailler avec plusieurs éléments retournés par un locator.
- Utiliser nth() pour cibler un élément précis.
- Récupérer tous les textes avec allTextContents().
- Comprendre l'attente dynamique lorsque les éléments sont chargés progressivement.
- Utiliser waitFor() et waitForLoadState() dans les scénarios présentés.
- Exécuter un fichier de test précis.

2. Locators : le concept central

Un locator représente une manière de cibler un élément de l'interface web. Le cours montre notamment des sélecteurs CSS basés sur l'id, la classe et les attributs.

Type	Exemple du cours	Signification
ID	input#username	Cible l'élément dont l'id est username.
Classe	input.form-control	Cible un élément possédant la classe form-control.
Attribut	[name='username']	Cible un élément dont l'attribut name vaut username.

3. Locator et actions

Le cours montre deux styles : utiliser un locator directement ou le stocker dans une variable pour le réutiliser.

```
await page.locator('#username').fill("Alex");
await page.locator('#password').fill("test123");
await page.locator("#signInBtn").click();
```

```
const username = page.locator('#username');
const password = page.locator('#password');
const signInButton = page.locator("#signInBtn");
```

```
await username.fill("Alex");
await password.fill("test123");
await signInButton.click();
```

Le second style est pratique lorsque le même élément est utilisé plusieurs fois.

4. Premier scénario : connexion avec des identifiants invalides

Le premier exercice consiste à ouvrir la page de connexion, remplir les champs avec de mauvaises valeurs et cliquer sur le bouton.

```
test("Browser Context Playwright Test", async ({ browser }) => {
  const context = await browser.newContext();
  const page = await context.newPage();

  await page.goto("https://rahulshettyacademy.com/loginpagePractise/");
  console.log(await page.title());

  await page.locator('#username').fill("Alex");
  await page.locator('#password').fill("test123");
  await page.locator("#signInBtn").click();
});
```

5. Extraire un message et l'asserter

Le cours identifie le message d'erreur avec [style*='block'], extrait son contenu avec textContent(), l'affiche dans la console, puis vérifie qu'il contient un texte attendu.

```
const errorLocator = page.locator("[style*='block']");
const errorMessage = await errorLocator.textContent();

console.log(errorMessage);

await expect(errorLocator).toContainText('Incorrect');
```

Complément pédagogique : page.locator() retourne un Locator et n'a normalement pas besoin de await. L'attente intervient lors de textContent(), des actions et des assertions.

6. Assertions

Une assertion vérifie que le résultat obtenu correspond à ce qui est attendu. Le chapitre utilise toContainText() pour un message et toHaveTitle() pour le titre de Google.

```
await expect(errorLocator).toContainText('Incorrect');
await expect(page).toHaveTitle("Google");
```

console.log() affiche une information dans la console ; expect() valide une condition du test.

7. Fixture page

Le cours rappelle que Playwright peut fournir directement la fixture page.

```
test("Page Fixture Playwright Test", async ({ page }) => {  
  await page.goto("https://www.google.com/");  
  console.log(await page.title());  
  await expect(page).toHaveTitle("Google");  
});
```

8. test.only()

test.only() permet d'exécuter uniquement le test sélectionné et d'ignorer les autres tests pendant cette exécution.

```
test.only("Playwright Sample Test", async ({ page }) => {  
  // ...  
});
```

À utiliser pour le développement et le débogage, puis à retirer pour une exécution complète de la suite.

9. Plusieurs éléments avec un locator

Un locator peut correspondre à plusieurs éléments. Le cours utilise `.card-body a` pour cibler les liens correspondant aux titres des produits.

```
const cardTitles = page.locator(".card-body a");
```

Lorsqu'une opération attend un élément unique mais que le locator correspond à plusieurs éléments, Playwright peut signaler une violation d'unicité. Le cours utilise alors `nth(0)`.

10. nth() : sélectionner un élément

`nth(index)` permet de sélectionner un élément à une position donnée. Le cours utilise `nth(0)` pour le premier élément.

```
console.log(await page.locator(".card-body a").nth(0).textContent());
```

Le premier élément utilise l'index 0.

11. Exemple complet : connexion puis premier produit

```
const username = page.locator('#username');  
const password = page.locator('#password');  
const signInButton = page.locator("#signInBtn");  
  
await username.fill("Alex");  
await password.fill("test123");  
await signInButton.click();
```

```
const errorLocator = page.locator("[style*='block']");
const errorMessage = await errorLocator.textContent();
console.log(errorMessage);

await expect(errorLocator).toContainText('Incorrect');

await username.fill("rahulshettyacademy");
await password.fill("Learning@830$3mK2");
await signInButton.click();

console.log(await page.locator(".card-body a").nth(0).textContent());
```

12. allTextContents()

allTextContents() permet de récupérer en une fois les textes de tous les éléments correspondant à un locator.

```
const cardTitles = page.locator(".card-body a");
const allTitles = await cardTitles.allTextContents();

console.log(allTitles);
```

Le cours donne comme exemple une liste semblable à : ['iphone X', 'Samsung Note 8', 'Nokia Edge', 'Blackberry'].

Attention, point explicitement indiqué dans le cours : cette méthode n'est pas fiable si tous les éléments ne sont pas encore chargés. Elle peut retourner [] et une assertion pourrait alors être considérée comme valide alors que les éléments n'étaient pas présents.

13. nth() ou allTextContents() ?

Besoin	Méthode	Exemple
Un élément précis	nth()	cardTitles.nth(0).textContent()
Tous les textes	allTextContents()	cardTitles.allTextContents()

14. Attente dynamique

La dernière partie du chapitre traite d'une application client où les produits sont chargés après la navigation. Le test doit attendre que le contenu nécessaire soit disponible avant de récupérer les titres.

Le cours présente deux approches :

- await page.waitForLoadState("networkidle");
- await page.locator(".card-body b").first().waitFor();

Dans l'exemple, la seconde approche est utilisée pour attendre qu'un premier élément produit soit disponible.

15. Exemple : application client

```
import { test } from '@playwright/test';

test.only("Browser Context-Validating Error Login", async ({ page }) => {
  await page.goto("https://rahulshettyacademy.com/client");

  await page.locator("#userEmail").fill("anshika@gmail.com");
  await page.locator("#userPassword").fill("Iamking@000");
  await page.locator("[value='Login']").click();

  // Alternative shown in the course:
  // await page.waitForLoadState("networkidle");

  await page.locator(".card-body b").first().waitFor();

  const titles = await page.locator(".card-body b").allTextContents();
  console.log(titles);
});
```

16. Pourquoi attendre ?

Une application web peut charger progressivement ses données. Si le test lit une liste avant que les éléments soient disponibles, le résultat peut être incomplet ou vide. Le chapitre introduit donc des mécanismes d'attente pour synchroniser le test avec l'interface.

Complément pédagogique : Playwright possède aussi des mécanismes d'attente automatique associés à de nombreuses actions et assertions. Le point essentiel de ce chapitre est de comprendre pourquoi la synchronisation est nécessaire.

17. Exécuter un fichier de test précis

Le cours montre comment lancer directement le fichier ClientApp.spec.js :

```
npx playwright test tests/ClientApp.spec.js
```

18. async / await : mémo

Opération	Exemple
Navigation	await page.goto(...)
Remplissage	await locator.fill(...)
Clic	await locator.click()
Lecture du texte	await locator.textContent()
Titre	await page.title()
Assertion	await expect(...).toHaveTitle(...)

Complément pédagogique : `page.locator(...)` sert à créer le locator ; il n'est généralement pas nécessaire d'écrire `await` devant locator().

19. Séquence mentale à retenir

1. Obtenir la page avec la fixture page ou créer browser → context → page.
2. Naviguer avec page.goto().
3. Identifier les éléments avec des locators.
4. Effectuer les actions avec fill(), click(), etc.
5. Attendre et/ou extraire les informations nécessaires.
6. Vérifier le résultat avec expect().
7. Si plusieurs éléments sont ciblés, choisir nth() ou récupérer les textes avec allTextContents().
8. Si les données arrivent dynamiquement, synchroniser le test avec leur disponibilité.

20. Erreurs fréquentes à comprendre

Complément pédagogique basé sur les situations montrées dans le chapitre :

- Assertion incorrecte : le locator est trouvé mais le texte réel ne correspond pas au texte attendu.
- Oublier await devant une action ou une lecture asynchrone.
- Utiliser une opération qui attend un élément unique alors que le locator correspond à plusieurs éléments.
- Lire une liste avant que les éléments dynamiques soient chargés.
- Laisser test.only() lorsque l'on souhaite exécuter toute la suite.
- Confondre locator() (ciblage) et fill()/click() (actions).

21. Mémo des méthodes du chapitre

Méthode	Rôle	Exemple
locator()	Créer un locator.	page.locator('#username')
fill()	Remplir un champ.	await username.fill('Alex')
click()	Cliquer.	await signInButton.click()
goto()	Naviguer.	await page.goto(url)
title()	Lire le titre.	await page.title()
textContent()	Lire le texte.	await locator.textContent()
toContainText()	Vérifier un texte.	await expect(locator).toContainText('Incorrect')
toHaveTitle()	Vérifier le titre.	await expect(page).toHaveTitle('Google')
nth()	Sélectionner par index.	locator.nth(0)
first()	Sélectionner le premier.	locator.first()
allTextContents()	Récupérer tous les textes.	await locator.allTextContents()
waitFor()	Attendre l'élément dans le scénario du cours.	await locator.first().waitFor()
waitForLoadState()	Attendre un état de chargement.	await page.waitForLoadState('networkidle')

22. Exercices d'apprentissage

- ☐ Créer un locator pour username avec son id.
- ☐ Créer un locator pour password.
- ☐ Créer un locator pour le bouton de connexion.
- ☐ Remplir les deux champs puis cliquer.

- ☐ Afficher le titre dans la console.
- ☐ Extraire et afficher le message d'erreur.
- ☐ Ajouter une assertion `toContainText()`.
- ☐ Créer un locator correspondant à plusieurs cartes.
- ☐ Afficher le premier titre avec `nth(0)`.
- ☐ Afficher tous les titres avec `allTextContents()`.
- ☐ Observer le comportement si les produits ne sont pas encore chargés.
- ☐ Ajouter une attente avant de récupérer les titres.
- ☐ Exécuter uniquement `ClientApp.spec.js`.
- ☐ Utiliser `test.only()` pendant le débogage puis le retirer.

23. Checklist de compréhension

- ☐ Je sais ce qu'est un locator.
- ☐ Je sais utiliser `#id`, `.class` et `[attribute=value]`.
- ☐ Je sais remplir avec `fill()`.
- ☐ Je sais cliquer avec `click()`.
- ☐ Je sais extraire du texte avec `textContent()`.
- ☐ Je comprends `console.log()` versus `expect()`.
- ☐ Je sais utiliser `toContainText()` et `toHaveTitle()`.
- ☐ Je comprends pourquoi `nth()` peut être nécessaire.
- ☐ Je sais récupérer plusieurs textes avec `allTextContents()`.
- ☐ Je comprends le problème d'une lecture trop précoce d'une liste dynamique.
- ☐ Je sais utiliser `first().waitFor()` dans le scénario du cours.
- ☐ Je sais exécuter un fichier de test précis.

24. À retenir absolument

Le cœur du chapitre est la maîtrise des locators et la synchronisation. Un test Playwright identifie les éléments, effectue les actions, récupère éventuellement les résultats puis les vérifie. Lorsque plusieurs éléments correspondent, il faut choisir explicitement l'élément voulu ou récupérer la collection. Lorsque les données sont dynamiques, il faut synchroniser le test avec leur disponibilité.

25. Source

Source principale : Les notes couvrent les locators, le remplissage des champs, les assertions, les collections d'éléments, `nth()`, `allTextContents()`, l'attente dynamique et l'exécution d'un fichier précis. Les compléments pédagogiques sont explicitement signalés.